Project Phase 3

Code of Conduct

All assignments are graded, meaning we expect you to adhere to the academic integrity standards of NYU Abu Dhabi. To avoid any confusion regarding this, we will briefly state what is and isn't allowed when working on an assignment. Any documents and program code that you submit must be fully written by yourself. You can discuss your work with fellow students, as long as these discussions are restricted to general solution techniques. Put differently, these discussions should not be about concrete code you are writing, nor about specific results you wish to submit. When discussing an assignment with others, this should never lead to you possessing the complete or partial solution of others, regardless of whether the solution is in paper or digital form, and independent of who made the solution, meaning you are also not allowed to possess solutions by someone from a different year or course, by someone from another university, or code from the Internet, etc. This also implies that there is never a valid reason to share your code with fellow students, and that there is no valid reason to publish your code online in any form. Every student is responsible for the work they submit. If there is any doubt during the grading about whether a student created the assignment themselves (e.g., if the solution matches that of others), we reserve the option to let the student explain why this is the case. In case doubts remain, or we decide to directly escalate the issue, the suspected violations will be reported to the academic administration according to the policies of NYU Abu Dhabi (see https://students.nyuad.nyu.edu/campus-life/community- standards/policies/academic-integrity/).

Objective

In this phase, you have to upgrade Phase2 server with multitasking capability using threads. The server should be able to serve several *n* clients simultaneously. Each thread should handle the request, communication and computation with one client.

The server terminal should print a message for every incoming and outgoing message, including the IP address and port number of each connected client, in the following required format (sample output for 3 clients for instance):

```
[INFO] Server started, waiting for client connections...
[INFO] Client #1 connected from 192.168.1.100:56789. Assigned to Thread-1.
[INFO] Client #2 connected from 192.168.1.101:56790. Assigned to Thread-2.
[INFO] Client #3 connected from 192.168.1.102:56791. Assigned to Thread-3.

[RECEIVED] [Client #2 - 192.168.1.101:56790] Received command: "unknowncmd"
[EXECUTING] [Client #2 - 192.168.1.101:56790] Executing command: "unknowncmd"
[ERROR] [Client #2 - 192.168.1.101:56790] Command not found: "unknowncmd"
[OUTPUT] [Client #2 - 192.168.1.101:56790] Sending error message to client:
"Command not found: unknowncmd"

[RECEIVED] [Client #3 - 192.168.1.102:56791] Received command: "pwd"
[EXECUTING] [Client #3 - 192.168.1.102:56791] Executing command: "pwd"
[OUTPUT] [Client #3 - 192.168.1.102:56791] Sending output to client:
/home/user

[RECEIVED] [Client #1 - 192.168.1.100:56789] Received command: "ls -l"
[EXECUTING] [Client #1 - 192.168.1.100:56789] Executing command: "ls -l"
[OUTPUT] [Client #1 - 192.168.1.100:56789] Sending output to client:
total 12
-rw-r--r--  1 user  user  1234 Feb 27 12:00 file1.txt
-rw-r--r--  1 user  user  5678 Feb 27 12:05 file2.txt

[RECEIVED] [Client #3 - 192.168.1.102:56791] Received command: "exit"
[INFO] [Client #3 - 192.168.1.102:56791] Client requested disconnect. Closing
connection.
[INFO] Client #3 disconnected.
```

Figure 1: Server Output

```
$ ls -l
total 12
-rw-r--r--  1 user  user  1234 Feb 27 12:00 file1.txt
-rw-r--r--  1 user  user  5678 Feb 27 12:05 file2.txt
```

Figure 2: Client #1 Output

```
$ unknowncmd
Command not found: unknowncmd
```

Figure 3: Client #2 Output

```
$ pwd
/home/user

$ exit
Disconnected from server.
```

Figure 4: Client #3 Output

Report Guidelines

Section	Description
Title Page	Includes the phase number, group member names, and NetIDs.
Architecture and Design	Describes the high-level structure of the system, key design decisions, reasons for using specific algorithms/data structures, file structure, and code organization.
Implementation Highlights	Explains the core functionalities, including important functions, algorithms, and logic. Includes references to critical code snippets. Also describes how errors and edge cases were handled.
Execution Instructions	Provides instructions on how to compile and run the program.
Testing	Lists the test cases performed. Includes screenshots and text explanations detailing the tests conducted and the output received.
Challenges	Describes any difficulties and challenges encountered during development and explains how they were resolved.
Division of Tasks	Clarify how the tasks/responsibilities were divided across both team members
References	Cites any resources used during development.

Grading Rubric

Description	Points (/15)
Successful compilation with a Makefile on remote Linux Server	1
Proper multithreaded execution of the commands by multiple clients (with server output format as required in given figures)	10
Error Handling	1
Detailed Comments	1
Code modularity, quality, efficiency and organization	1
Report	1

Noteworthy Points

- Each phase is cumulative: all functionality from previous phases must persist and remain correct.

- Feedback comments from previous phases must be addressed; unresolved issues will carry forward as bugs and lead to repeated deductions.

- Although it is understood that you may exchange ideas on how to make things work and seek advice from your fellow students, sharing of code is not allowed.

- If you use code that is not your own, you will have to provide appropriate citation (i.e., explicitly state where you found the code). Otherwise, plagiarism questions may ensue. Regardless, you have to fully understand what and how such pieces of code do.

- Make sure to use separate compilation and apply best practices in your code submission. Aim for high efficiency, reduction of redundant code, and scalability to facilitate updates in future phases.

- Extensive and proper commenting must be done for all the program. Avoid short comments before huge code blocks. They should be meaningful, detailed and explaining all your code logic.

- Your submission **must** be working on the **remote Linux server** successfully.

- Submissions:
  A .zip file containing the following:
  - C files + Make file
  - Report (Must be in PDF format)